Распределённая файловая система HDFS

Марковский Артём Владиславович, 466636, J3310

Развёртывание кластера

Команда запуска

```
$r = "$PWD\rustfs"
docker run -d --name rustfs -p 9000:9000 -p 9001:9001 `
  -e RUSTFS_ACCESS_KEY=rustfsadmin -e RUSTFS_SECRET_KEY=rustfsadmin `
  -e RUSTFS_VOLUMES="/data/disk{1...8}" `
  -e RUSTFS_ADDRESS=":9000" -e RUSTFS_CONSOLE_ADDRESS=":9001" `
  -e RUSTFS_CONSOLE_ENABLE=true `
  -e RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true `
  -v "${r}\disk1:/data/disk1" ... -v "${r}\disk8:/data/disk8" `
  rustfs/rustfs:latest
```

Первая попытка: ошибка проверки дисков

С первого раза у меня RustFS не запустился. Сервер увидел, что все 8 папок сидят на одном устройстве (st_dev='0:66'), а для erasure-эндпоинтов ему нужны разные физические диски. Для локального теста эту проверку проверку я выключил, добавив в команду запуска переменную окружения RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true.

```
e69fdf0542e2: Pull complete
Digest: sha256:1803faef57627e2d9c2e7d89d655d712ddded53890400549871630 43fecb6a3c
Status: Downloaded newer image for rustfs/rustfs:latest
fec4bf13ec60d1252636382c7435d288c530706cc78f7303a42f0ca75c86b453
PS C:\python\Study\ITMO_3nd\EngData> docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS          PORTS     NAMES
7adc1b557865   aquasec/trivy:latest   "cat /usr/local/bin/…"   25 seconds ago   Up 24 seconds             quizzical_hawk
ing
PS C:\python\Study\ITMO_3nd\EngData> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\python\Study\ITMO_3nd\EngData> docker logs rustfs --tail 40
WARNING: RUSTFS_ACCESS_KEY uses the default rustfsadmin credential. Set non-default credentials for production deploymen
ts; with an all-default pair, multi-node clusters additionally need RUSTFS_RPC_SECRET to derive internode RPC auth.
WARNING: RUSTFS_SECRET_KEY uses the default rustfsadmin credential. Set non-default credentials for production deploymen
ts; with an all-default pair, multi-node clusters additionally need RUSTFS_RPC_SECRET to derive internode RPC auth.
Initializing data directories: /data/disk1 /data/disk2 /data/disk3 /data/disk4 /data/disk5 /data/disk6 /data/disk7 /data
/disk8
Initializing log directory: /logs
Starting: /usr/bin/rustfs  /data/disk1 /data/disk2 /data/disk3 /data/disk4 /data/disk5 /data/disk6 /data/disk7 /data/dis
k8
[FATAL] Server runtime failed: local erasure endpoints must use distinct physical disks; detected shared devices [0:66 =
> /data/disk1, /data/disk2, /data/disk3, /data/disk4, /data/disk5, /data/disk6, /data/disk7, /data/disk8]. validation di
agnostics: [path='/data/disk1', canonical='/data/disk1', st_dev='0:66', device_ids=[0:66]; path='/data/disk2', canonical
='/data/disk2', st_dev='0:66', device_ids=[0:66]; path='/data/disk3', canonical='/data/disk3', st_dev='0:66', device_ids
=[0:66]; path='/data/disk4', canonical='/data/disk4', st_dev='0:66', device_ids=[0:66]; path='/data/disk5', canonical='/
data/disk5', st_dev='0:66', device_ids=[0:66]; path='/data/disk6', canonical='/data/disk6', st_dev='0:66', device_ids=[0
:66]; path='/data/disk7', canonical='/data/disk7', st_dev='0:66', device_ids=[0:66]; path='/data/disk8', canonical='/dat
a/disk8', st_dev='0:66', device_ids=[0:66]]. Set RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true only for local testing or CI to by
pass this safety check
PS C:\python\Study\ITMO_3nd\EngData> docker ps -a
```

Рис. 1: Ошибка shared devices при первом запуске

Результат

Со второго раза контейнер поднялся: S3 API на порту 9000, веб-консоль на 9001.

```
PS C:\python\Study\ITMO_3nd\EngData> docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED         STATUS         PORTS
                                       NAMES
65f0e79b46b4   rustfs/rustfs:latest   "/entrypoint.sh rust…"   8 seconds ago   Up 7 seconds   0.0.0.0:9000-9001->9000-90
01/tcp, [::]:9000-9001->9000-9001/tcp   rustfs
PS C:\python\Study\ITMO_3nd\EngData>
```

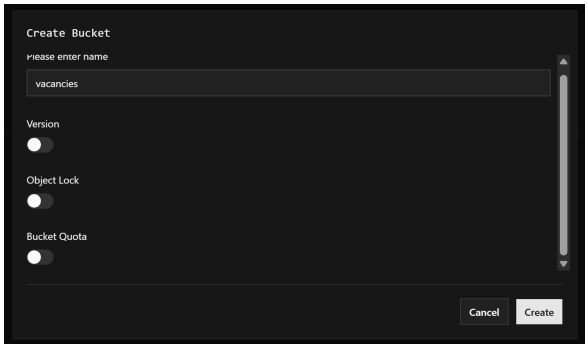
Рис. 2: Контейнер rustfs в статусе Up

Загрузка данных

В бакет vacancies я залил два набора:

- JSON-файлы вакансий hh.ru: 4 284 файла, 23 963 663 байт (≈22,9 МиБ), префикс hh/;
- датасет ИТ-вакансий vacancies_2024.csv: 24 970 431 974 байт (≈23,3 ГиБ), префикс big/.

Оба грузил через веб-консоль. С JSON загрузка затянулась: браузер держит не больше 6 соединений к одному серверу. Большой CSV консоль сама порезала на части по 16 МиБ (multipart upload).



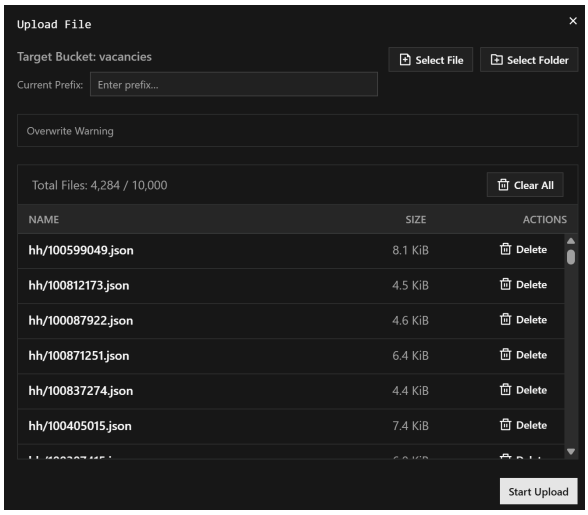


Рис. 3: Создание бакета и загрузка 4 284 JSON-файлов

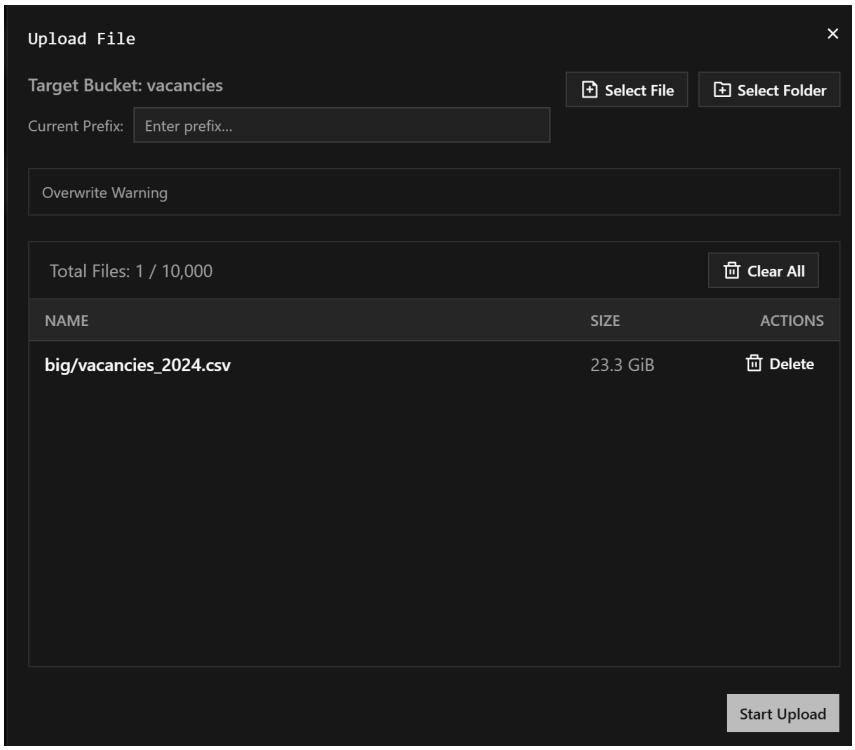


Рис. 4: Загрузка vacancies_2024.csv (23,3 ГиБ)



Рис. 5: Рост папок-дисков во время загрузки большого файла

Размеры папок-дисков

Размер и число файлов на каждом диске считал так:

```
1..8 | ForEach-Object {
  $f = Get-ChildItem ".\rustfs\disk$_" -Recurse -File -Force
  [PSCustomObject]@{ Disk="disk$_"; Files=$f.Count;
    MB=[math]::Round(($f | Measure-Object Length -Sum).Sum/1MB,1) }
} | Format-Table -AutoSize
```

	Только JSON		JSON + CSV	
Папка	Файлов	Размер, КБ	Файлов	Размер, МиБ
disk1–disk8 (каждая)	4 310	8 588,8	7 283	5 962,6

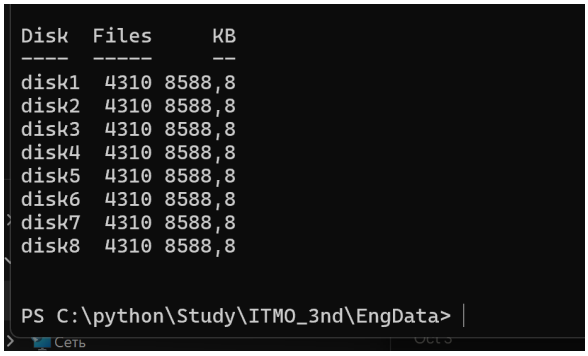
Таблица 1: Размеры папок-дисков

RustFS разложил данные по дискам поровну: на всех восьми одинаковое число файлов и одинаковый объём.

Набор	Исходный размер	На 8 дисках	Коэффициент
JSON (4 284 файла)	23,96 МБ	70,36 МБ	2,94
JSON + CSV	24,99 ГБ	50,02 ГБ	2,001
только CSV	24,97 ГБ	49,95 ГБ	2,0003

Таблица 2: Фактический коэффициент избыточности

На крупном файле коэффициент почти ровно теоретический, $(k + m)/k = 2$. А вот на мелких JSON он подбирается к 3. Метаданные весят около 566 байт на объект (подробнее в разделе о фрагментах), это сравнимо с самим JSON, и копия метаданных лежит на каждом из 8 дисков. Чтобы это поправить, мелкие файлы можно объединить в один архив, как советовали на практическом занятии.



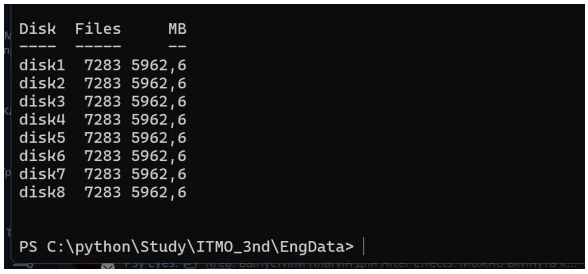


Рис. 6: Размеры дисков после загрузки JSON (слева) и после загрузки CSV (справа)

Как файл делится на фрагменты

Крупный объект

На каждом диске объект big/vacancies_2024.csv выглядит как папка, но ей не является (так как это S3). Внутри xl.meta (73 379 байт, описание объекта и всех его частей) и каталог данных с файлами part.N и part.N.meta. Как я понял, part.N.meta нужны просто для мульпарт загрузки: после удаления они не появлялись заново, когдая пытался восстановить, и всё равно всё работало.

vacancies_2024.csv
24 970 431 974 байт

↓

multipart upload частями по 16 МиБ

↓

1 489 частей
1 488 по 16 777 216 байт + последняя 5 934 566 байт

↓

каждая часть: 4 data-фрагмента + 4 parity-фрагмента

↓

part.N на каждом из 8 дисков

4 194 816 байт = 16 МиБ / 4 + 512 байт

Смотреть надо на размер part.N: 4 194 816 байт. Это ровно четверть части (4 194 304 байт) плюс 512 байт. Выходит, каждая часть режется на $k = 4$ data-фрагмента, а ещё 4 диска держат parity ($m = 4$). Лишние 512 байт занимают контрольные суммы. Часть в 16 МиБ кодируется блоками по 1 МиБ, от каждого блока на диск попадает фрагмент в 256 КиБ, и перед каждым фрагментом RustFS пишет 32 байта HighwayHash-256: $16 \times 32 = 512$. Формула размера есть в исходниках RustFS 1.0.1 (функция bitrot_shard_file_size в crates/ecstore/src/erasure/coding/bitrot.rs), а на тестовом объекте я пересчитал хэши всех блоков, и они совпали.

Число файлов тоже сходится: на каждом диске у объекта один xl.meta и по два файла на каждую часть (part.N и part.N.meta), то есть $1 + 2 \times 1\,489 = 2\,979$.

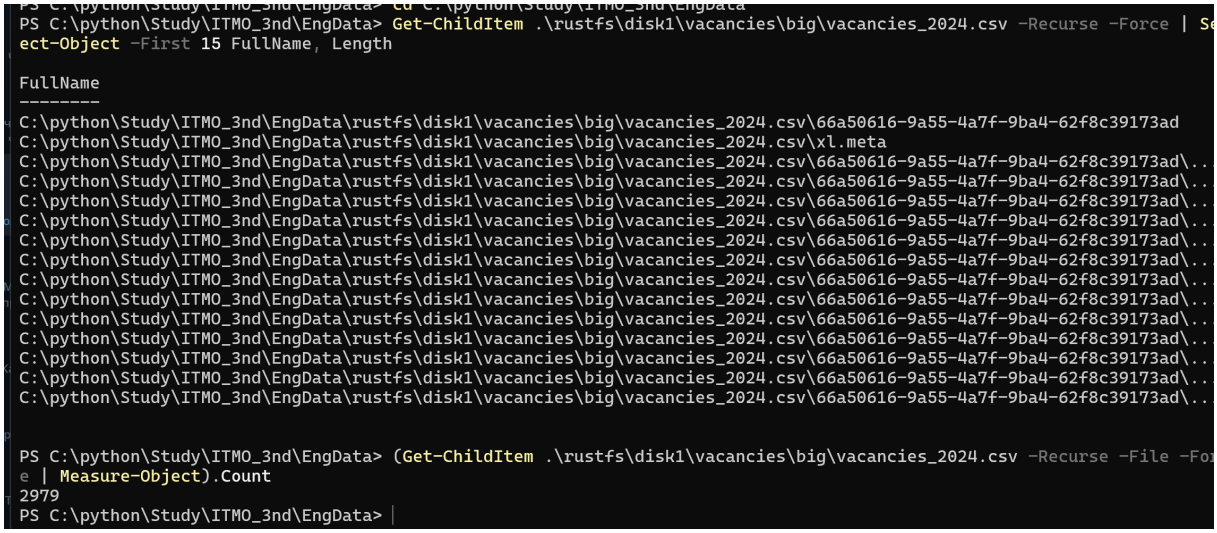


Рис. 7: Структура объекта на disk1: xl.meta и каталог данных

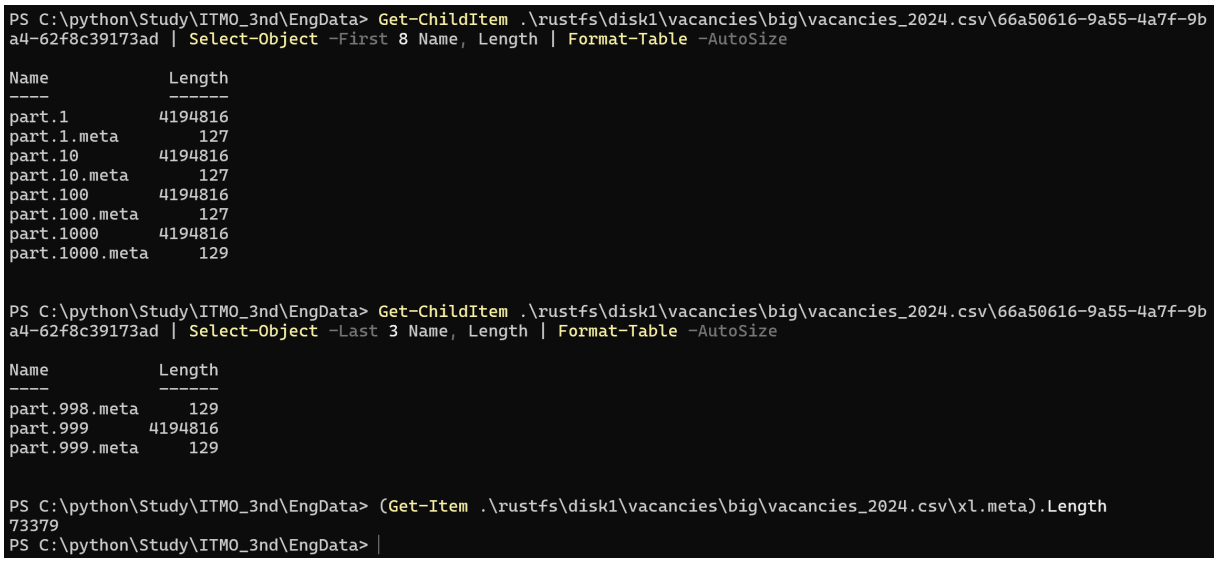


Рис. 8: Размеры part.N, part.N.meta и xl.meta

Мелкие объекты

С мелкими объектами картина другая. JSON hh/100087922.json весит 4 726 байт, и на каждом из 8 дисков от него остаётся один xl.meta на 1 780 байт. Объект меньше блока в 1 МиБ, поэтому RustFS не заводит для него каталог данных, а кладёт фрагменты прямо внутрь xl.meta.

Отсюда и коэффициент 2,94 на мелких файлах. Полезных данных на 8 дисках $8 \times 1\,182 \approx 2 \times 4\,726$, как и положено для 4+4. Но к каждому фрагменту добавляются хэш и полкилобайта метаданных, и объём вырастает ещё в полтора раза: $8 \times 1\,780 = 14\,240$ байт, втрое больше самого файла.

Сценарии отказа

Потеря одной папки

Удалил содержимое disk1. Бакет при этом никуда не делся: 4 285 объектов, 23,3 ГиБ. RustFS сам пометил диск как восстанавливаемый (появился файл .rustfs.sys/healing.bin) и начал пересчитывать фрагменты из оставшихся семи.

Наблюдения о восстановлении (heal)

Большой объект RustFS собирает во временной папке .rustfs.sys/tmp и переносит в бакет только целиком. Писал он примерно 5,7–7 МБ/с на диск. Первая попытка оборвалась на 349-й части из 1 489 (1,4 ГБ): восстановление встало, а временные данные сервер выкинул. После повторной замены диска CSV вернулся полностью (все 1 489 частей), а сам диск дорос до 5 956,3 МБ против 5 962,6 МБ на исправном.

С мелкими JSON хуже. Статус восстановления несколько раз уходил в Idle, хотя диск ещё не заполнился: примерно через час на disk1 было 2 937 JSON из 4 284. Я запустил heal руками (rc admin heal start): он прошёл 1 052 объекта, 1 051

починил, на одном споткнулся и остановился. Повторный запуск сервер отклонил с причиной overlapping_paths.

Зато обычное чтение объекта по ключу (GET) чинило его сразу: недостающие фрагменты тут же появлялись на пустых дисках (read repair).

Полез искать, бывает ли такое у других. В issue RustFS #1533 люди жалуются ровно на это в той же конфигурации (один узел и несколько дисков в контейнере), в #1758 на похожее в кластере. Поэтому думаю, что это баг самого RustFS, и судя по #1533, проявляется он как раз, когда все диски на одной машинке, как у меня.

Потеря 50% папок (4 из 8)

Удалил папки disk1–disk4 целиком, и консоль показала их в статусе Unknown. Создал пустые папки заново, перезапустил контейнер: все 8 дисков снова Online.

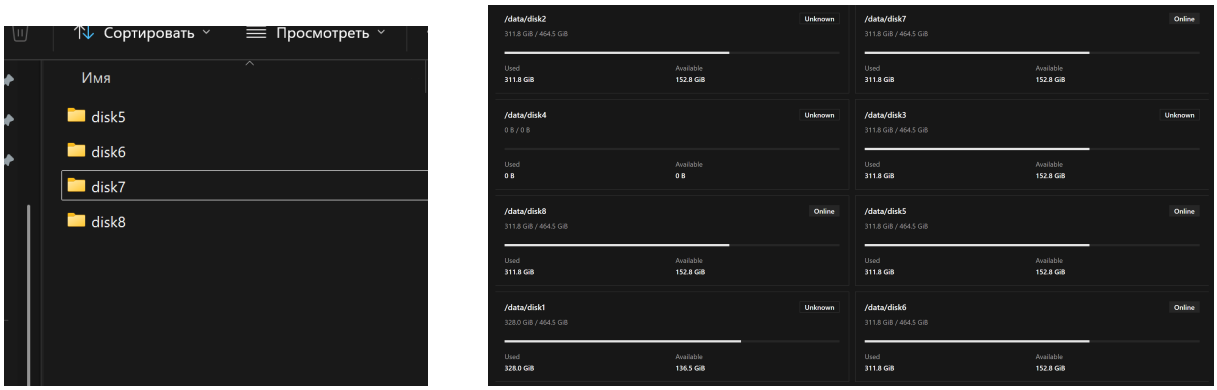


Рис. 9: Удалены disk1–disk4: диски в статусе Unknown

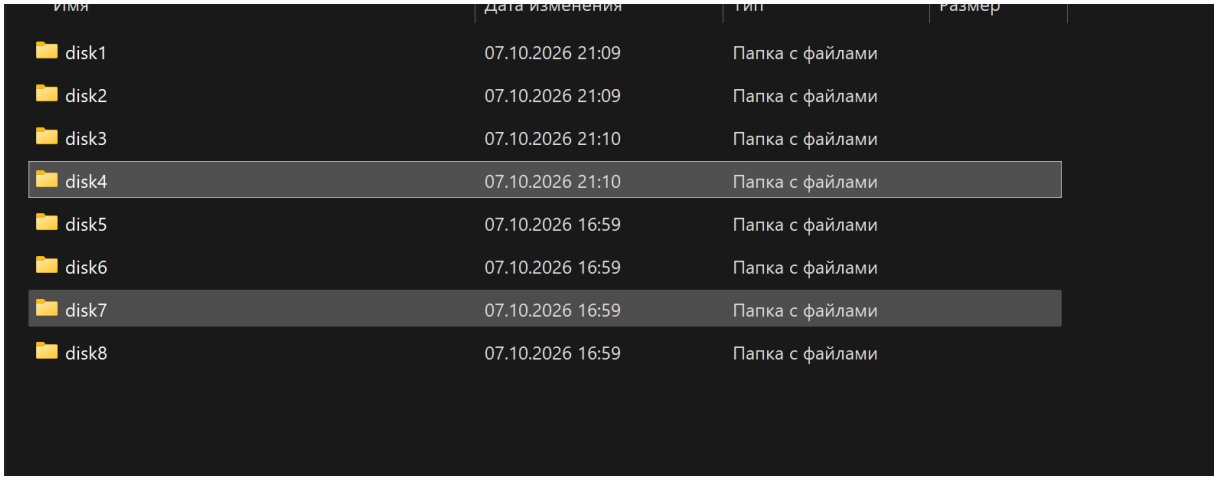


Рис. 10: Папки disk1–disk4 созданы заново

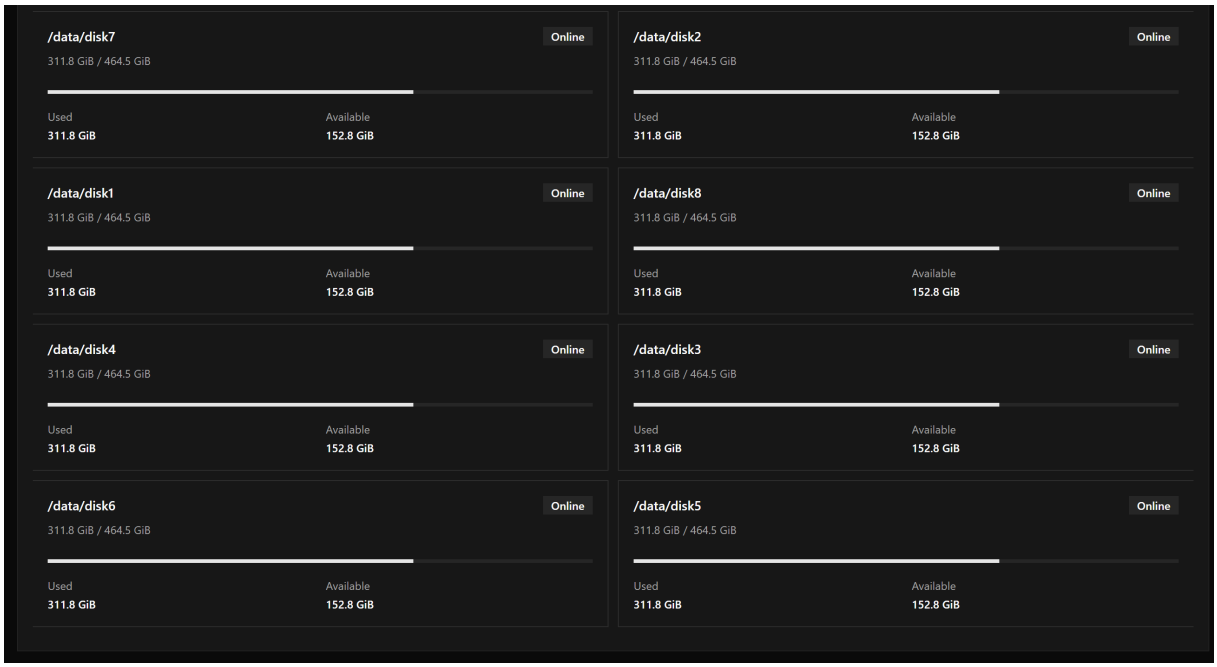


Рис. 11: Все 8 дисков снова Online, данные только на 4 из них

Чтение по ключу. Читал через rc прямо в контейнере:

```
docker exec rustfs sh -c 'cd /tmp;
  ./rc cp local/vacancies/hh/100087922.json s4.json; sha256sum s4.json;
  ./rc cat local/vacancies/big/vacancies_2024.csv | head -c 10485760 | sha256sum'
```

Объект	Прочитано из RustFS	Оригинал
100087922.json	1b7f6735…aadc6739	1b7f6735…aadc6739
первые 10 МБ CSV	3127147b…07eb7273e	3127147b…07eb7273e

Таблица 3: SHA-256 при потере 4 из 8 дисков: хэши совпадают

Хэши совпали, данные целы. А сразу после чтения 100087922.json появился на всех четырёх пустых дисках (xl.meta, 1 780 байт).

Листинг бакета. А вот веб-консоль бакет открыть не смогла. Сначала запрос ListObjectsV2 вернул пустой список (KeyCount=0), позже начал отвечать 503 SlowDownRead («Resource requested is unreadable»). И это при том, что по ключу всё читается. Как я это понимаю, листинг опрашивает только часть дисков, и если попадает на пустые, но формально исправные, видит пустой бакет. Опять же связываю это с тем, что всё проводил на одной машинке.

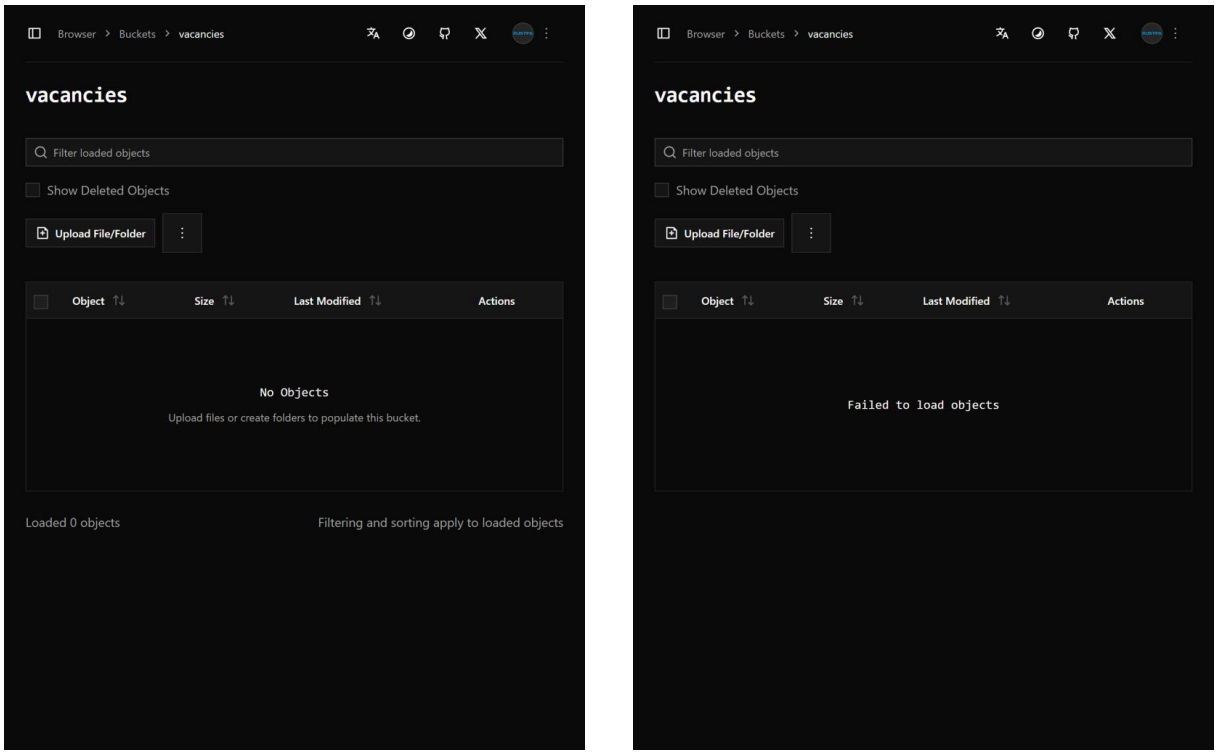


Рис. 12: Листинг бакета при потере 50%: пустой список и ошибка 503

Потеря 50%+1 папки (5 из 8)

Удалил ещё и disk5.

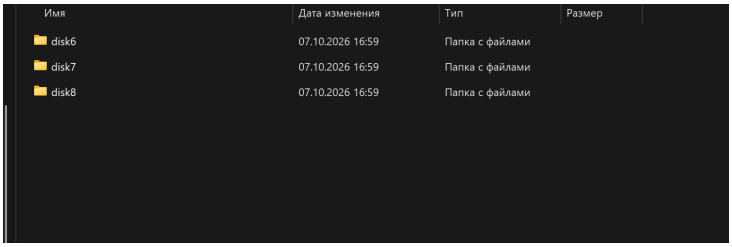


Рис. 13: Остались только disk6–disk8

В веб-консоль с теми же ключами rustfsadmin больше не пускает («Login Failed»). Чтение через rc и для JSON, и для CSV падает с 403 AccessDenied («Authentication failed»). Фрагментов осталось 3 из 8, а для сборки нужно минимум 4, поэтому данные не читаются. Почему сервер отвечает ошибкой авторизации, а не ошибкой чтения, я так и не выяснил: rustfsadmin задан переменной окружения, и читать его с дисков серверу не нужно.

```
PS C:\Users\shlaapnik> docker exec rustfs sh -c 'cd /tmp; ./rc cp local/vacancies/hh/102087778.json s5.json; sha256sum s
5.json; ./rc cat local/vacancies/big/vacancies_2024.csv | head -c 10485760 | sha256sum'
✗ Failed to download local/vacancies/hh/102087778.json: Authentication failed: Service error: unhandled error (AccessDen
ied) (status: 403, code: AccessDenied)
sha256sum: s5.json: No such file or directory
✗ Failed to read local/vacancies/big/vacancies_2024.csv: Authentication failed: Service error: unhandled error (AccessDe
nied) (status: 403, code: AccessDenied)
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  -
PS C:\Users\shlaapnik>
```

Рис. 14: Чтение при потере 5 из 8 дисков: 403 AccessDenied

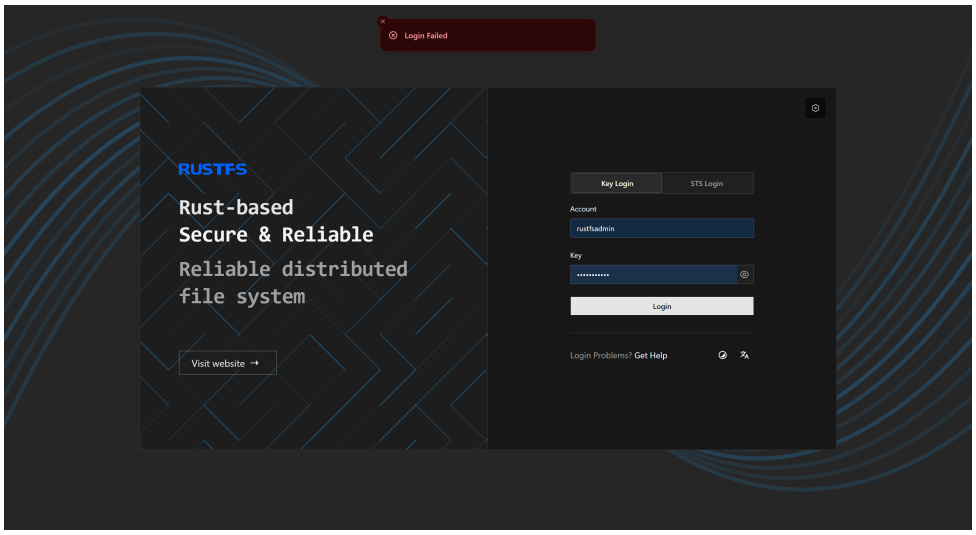


Рис. 15: Вход в консоль невозможен

Расчёт стоимости

Коэффициент избыточности считаю как отношение физического объёма к полезному:

$$K_{EC} = \frac{k+m}{k} = \frac{4+4}{4} = 2{,}0, \qquad K_{EC\,12+4} = \frac{12+4}{12} \approx 1{,}33, \qquad K_{rep} = 3{,}0.$$

Физический объём $V_{phys} = K \cdot V$.

Объём данных	EC 4+4 (×2)	EC 12+4 (×1,33)	3 реплики (×3)	Экономия 4+4
25 ГБ	50 ГБ	33,3 ГБ	75 ГБ	25 ГБ (33%)
50 ТБ	100 ТБ	66,7 ТБ	150 ТБ	50 ТБ (33%)

Таблица 4: Физический объём хранения; экономия EC 4+4 считается относительно трёх реплик

Обоснование решения

Выбор хранилища

Задача такая: хранить датасет вакансий (сейчас около 25 ГБ, дальше до 50 ТБ) и отдавать его аналитике. Я бы выбрал объектное хранилище с S3-интерфейсом и Erasure Coding. Почему:

- **Стоимость.** Коэффициент 2,0 против 3,0 у трёх реплик.
- **Надёжность.** При потере 4 дисков из 8 объекты читаются по ключу и хэши совпадают, при потере 5 данных нет. Три реплики переживают потерю только 2 копий.

- **Совместимость.** S3 API понимают Spark, pandas, DuckDB, Trino, polars и SDK всех основных языков: для объектных хранилищ это стандарт де-факто.

Для 50 ТБ я бы взял схему пошире, например EC 12+4 на 16 дисках в 4 серверах. Коэффициент $\approx 1{,}33$, это 66,7 ТБ физического объёма. Такая схема переживает потерю 4 дисков из 16, то есть одного сервера целиком.

Альтернативное решение

HDFS с репликацией $\times 3$. Плюсы у неё такие: восстановление простое, блок копируется с соседнего узла без пересчётов. Чтение при отказе диска не тормозит. И есть локальность данных: MapReduce или Spark запускают вычисления прямо рядом с блоками. Минусов два: дисков в полтора раза больше, чем у EC 4+4, и отдельный NameNode, который надо резервировать. В Hadoop 3 есть и свой Erasure Coding, но S3-интерфейса у него нет.

Ограничения и главный компромисс

Треть дисков экономим, отказов переживаем больше, работать с хранилищем через S3 API проще, а расплачиваемся скоростью восстановления. Чтобы вернуть потерянный фрагмент, сервер читает четыре других фрагмента и пересчитывает недостающий. У меня это шло со скоростью 5,7–7 МБ/с на диск, и часть мелких файлов вернулась только после чтения по ключу.